SNeuron: Learnable Structural Neuron Gating for Energy-Efficient Embedded AI

Mohammed S. Alziyad**, Mohammed Hassan Al-Zahrani**, Romina S. Molina*†‡ *Member, IEEE*,
Maria Liz Crespo‡, Marco Zennaro‡ *Senior Member, IEEE*
*KAUST: King Abdullah University of Science and Technology
Email: {m.alziyad911, Mohammed7Zahrani}@gmail.com
†KaleidoForge, Trieste, Italy
Email: rsmolina@kaleidoforge.com
‡The Abdus Salam International Centre for Theoretical Physics, Trieste, Italy
Email: {rmolina, mcrespo, mzennaro}@ictp.it

***Abstract*—Efficient deployment of deep neural networks on resource-constrained edge platforms remains challenging due to excessive parameterization and multi-stage compression pipelines. This work introduces *SNeuron*, a learnable neuron gating mechanism that enables simultaneous training and structural compression via a sparsity-inducing objective, *SLoss*. Unlike conventional pruning and quantization workflows, the proposed framework discovers compact task-relevant subnetworks in a single end-to-end process. Experimental evaluation across standard architectures demonstrates substantial reductions in parameter count and model size while preserving accuracy. Deployment on an FPGA for gamma–neutron discrimination further confirms competitive classification performance alongside significant hardware savings, including marked DSP reduction. These results position *SNeuron* as an effective pathway toward energy-efficient edge inference without complex compression pipelines.**

***Index Terms*—Machine Learning, Compression, Edge AI, Sparsity**

I. Introduction

Deep neural networks dominate state-of-the-art performance across a wide range of AI applications [1]. However, their deployment on resource-constrained devices remains challenging due to high computational complexity, memory footprint, and energy consumption. Large-scale architectures such as VGG and ResNet require hundreds of megabytes of storage and billions of floating-point operations per inference [2], [3], while lightweight models including MobileNet and ShuffleNet reduce, but do not eliminate, these constraints for embedded platforms [4]–[6].

Model compression [7] is essential for constrained edge deployment [8], [9], with pruning-based approaches removing redundant weights, neurons, or filters through iterative train–prune–retrain procedures [10].

Deep Compression further combines pruning, quantization, and entropy coding to achieve order-of-magnitude reductions in storage without accuracy loss [11]. The Lottery Ticket Hypothesis later revealed that dense networks inherently contain sparse subnetworks capable of matching full-model performance when properly initialized and trained [12].

Despite these advances, most existing approaches rely on multi-stage post-training pipelines or heuristic subnetwork discovery, limiting their efficiency and adaptability in embedded deployments. Recent work has explored single-stage compression approaches combining knowledge distillation with quantization-aware pruning [13], as well as differentiable and single-cycle pruning methods that reduce training overhead [14], [15]. However, structural sparsity in these approaches is either externally imposed or dependent on weight-derived saliency criteria, rather than intrinsically learned during optimization. These limitations motivate the development of learning mechanisms capable of discovering compact, task-relevant subnetworks directly within gradient-based training.

To mitigate these limitations, this work introduces *SNeuron*, a learnable neuron gating mechanism that enables neural networks to identify informative neurons and form compact task-relevant subnetworks during training. By integrating sparsity discovery directly into gradient-based optimization, the proposed framework performs model training and structural compression simultaneously. This allows one to avoid multi-stage pruning and fine-tuning while remaining compatible with resource-constrained embedded inference.

The main contributions of this work are summarized as follows:

1) We introduce *SNeuron*, a neuron model with a learnable gating parameter S that enables dynamic identification of informative neurons and promotes structured sparsity during standard gradient-based training.
2) Through the proposed *SLoss* objective, the framework performs simultaneous model training and compression, eliminating the need for post-training pruning or fine-tuning while preserving predictive accuracy and reducing memory and computational cost.
3) Experimental validation across four architectures and a real-world gamma/neutron discrimination task demonstrates up to $88\times$ compression with 97% accuracy retention, and FPGA deployment achieving a $3.5\times$ latency reduction and $62\times$ DSP reduction relative to quantization-aware baselines.

*These authors contributed equally to this work.

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.
Manuscript received April 19, 2021; revised August 16, 2021.

II. Emergent Minimal Architectures via Learnable Gates

Each neuron is associated with a learnable structural parameter S that controls its participation in forward inference. Neuron influence is governed by the continuous–binary gating mechanism proposed in Eq. 1, where $\sigma(\cdot)$ denotes the sigmoid function and $\tau = 0.5$ is a fixed threshold. Here, $g \in (0, 1)$ is the continuous gate activation and $T \in \{0, 1\}$ is the binary gate that determines neuron participation during inference.

$$g = \sigma(S), \qquad T = \mathbf{1}_{\{g>\tau\}}, \quad T \in \{0, 1\}, \tag{1}$$

This learnable gating mechanism enables the network to automatically identify a minimal subset of informative neurons during training, allowing the effective sparse architecture to emerge directly from optimization rather than manual design or post-training pruning. Because only neurons with $T = 1$ participate in inference, computational complexity scales with the number of selected neurons rather than the full network width, reducing multiply–accumulate operations, memory access, and energy consumption.

III. SNeuron

A conventional neuron computes

$$y = \phi(\mathbf{w}^\top \mathbf{x}),$$

where $\mathbf{w} \in \mathbb{R}^n$ denotes the weight vector, $\mathbf{x} \in \mathbb{R}^n$ the input vector, and ϕ the activation function.

SNeuron applies the gating mechanism defined in (1) to the neuron output, as presented in Eq. 2, where $\tilde{y}$ denotes the gated output and T is the binary gate defined in (1).

$$\tilde{y} = T \cdot \phi(\mathbf{w}^\top \mathbf{x}) \tag{2}$$

To enable gradient-based optimization despite the non-differentiable threshold, training employs a straight-through estimator (STE): the forward pass uses the binary gate T, while gradients are propagated through the continuous sigmoid function $\sigma(S)$. Applying *SNeuron* across layers promotes the emergence of compact subnetworks in which informative neurons remain active while redundant neurons are suppressed. This mechanism induces *structured sparsity* at the neuron or channel level, effectively reducing downstream dimensionality and required multiply–accumulate operations. In contrast to unstructured weight pruning, which preserves the computational graph and limits hardware acceleration, structured gating directly reduces inference cost.

Fig. 1 illustrates the proposed neuron formulation, where a learnable gating parameter modulates the output of each activation unit, allowing the network to dynamically suppress redundant neurons and form compact task-relevant subnetworks during training.

IV. SLoss

Gate parameters are optimized through a composite objective presented in Eq. 3, where $\mathcal{L}_{\text{task}}$ denotes the task-specific loss (e.g., cross-entropy), $g_i = \sigma(S_i)$ is the continuous gate defined in (1), N is the number of gated neurons, and λ controls the accuracy–sparsity trade-off.

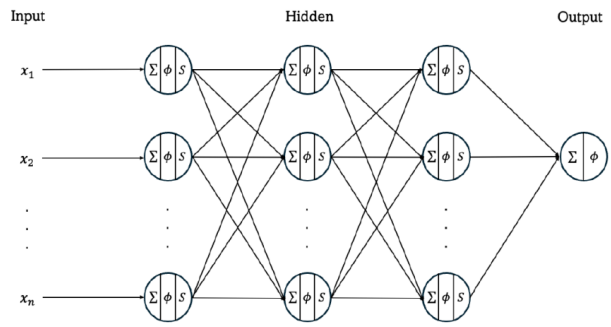

Fig. 1. Conceptual illustration of the proposed *SNeuron* mechanism. Each neuron augments the conventional weighted summation (Σ) and activation function (ϕ) with a learnable gating parameter S. This parameter controls neuron participation during forward inference, enabling structured sparsity across network layers.

$$\mathcal{L}_S = \mathcal{L}_{\text{task}} + \lambda \frac{1}{N} \sum_{i=1}^{N} g_i, \tag{3}$$

The regularization term corresponds to the expected fraction of active neurons and promotes structured sparsity during training, while binary gates are applied in forward inference via the straight-through estimator.

V. Structural Pruning and Deployment

Following training with *SLoss*, the learned gate parameters determine a sparse subnetwork through structural pruning. Neurons or channels for which $T = 0$ (i.e., $\sigma(S_i) \leq 0.5$) are permanently removed from the computational graph, along with their associated weights. The pruning procedure propagates structural changes across consecutive layers: removed output channels in a convolutional layer are reflected as removed input channels in the subsequent layer, and analogously for fully connected layers. This ensures that the pruned model constitutes a valid, self-consistent architecture without requiring fine-tuning or retraining.

Once pruning is complete, the *SNeuron* gating layers are eliminated from the model, since their role (identifying informative neurons) has been fulfilled during training.

The sparsity–accuracy trade-off is controlled entirely by the single hyperparameter λ in (3). Larger values of λ promote stronger sparsity at the cost of potential accuracy reduction, while smaller values preserve accuracy with less aggressive compression.

VI. Analytical Insight

A. Sparsity Induction via Gate Regularization

Let S_i denote the learnable parameter of neuron i, and let $g_i = \sigma(S_i) \in (0, 1)$ be the corresponding continuous gate activation. The sparsity regularizer is defined through Eq. 4, which represents the expected fraction of active neurons.

$$R_S = \frac{1}{N} \sum_{i=1}^{N} g_i, \tag{4}$$

Minimizing (3) therefore promotes solutions with fewer effectively active neurons, yielding structured sparsity at the neuron level. Increasing λ biases optimization toward configurations in which only neurons contributing significantly to task-loss minimization retain high gate activations, while redundant neurons converge toward zero.

B. Accuracy Preservation of Discovered Subnetworks

Let θ^* denote the optimal dense parameters minimizing the task loss, and let $\tilde{\theta}$ denote parameters obtained under *SLoss* with $k \ll N$ effective neurons. Because neurons with small gate activations contribute negligibly to forward computation, the resulting model can be interpreted as a sparse subnetwork of the dense architecture.

Assuming local smoothness of the task loss around θ^*, there exists a sparse parameterization $\tilde{\theta}$ such that

$$T_{\text{original}}(\tilde{\theta}) \leq T_{\text{original}}(\theta^*) + \epsilon, \tag{5}$$

for small $\epsilon > 0$ determined by λ and optimization dynamics. This behavior is consistent with the lottery ticket hypothesis [12].

C. Computational Efficiency

Let k denote the number of effectively active neurons after training. Forward inference complexity therefore scales with k rather than N, reducing multiply–accumulate operations, memory accesses, and energy consumption. Consequently, the proposed framework provides a direct mechanism for trading predictive accuracy against computational cost through the single hyperparameter λ, making it suitable for resource-constrained embedded deployment.

VII. Experimental Results and Analysis

The proposed *SNeuron* framework was evaluated on four neural network architectures: LeNet [16] and an ad-hoc TinyCNN on MNIST [17], VGG16 on CIFAR-10 [18], and an ad-hoc 1D-Convolutional Neural Network (1DCNN-GN) for real-world FPGA deployment in gamma/neutron discrimination [19]. Performance was assessed in terms of predictive accuracy, parameter count, model storage size, compression ratio, and accuracy retention, metrics that directly reflect suitability for resource-constrained embedded deployment.

A. Compression Performance

We compare *SNeuron* with a single-stage compression baseline based on knowledge distillation combined with quantization-aware pruning (KD+QAP). While this approach integrates compression within a unified training process, it still relies on externally guided structural adaptation via teacher supervision and pruning heuristics. In contrast, *SNeuron* learns its sparse structure intrinsically through differentiable gating, enabling competitive accuracy and hardware efficiency without distillation or explicit pruning procedures. KD+QAP results are reported where the distillation pipeline was available.

TABLE I
Model compression results using the proposed SNeuron method.

Model	Accuracy (%)	Parameters	Size (KB)	Comp. Ratio	Acc. Ret. (%)
LeNet (baseline)	98.13	44,426	170.00	–	–
LeNet + KD+QAP	91.72	2,823	11.03	15.7×	93.50
LeNet + S (mild)	98.08	3,477	13.00	13×	99.95
LeNet + S (strong)	95.25	505	1.97	88×	97.06
TinyCNN (baseline)	98.54	33,194	132.80	–	–
TinyCNN + KD+QAP	86.96	934	3.64	35.5×	88.10
TinyCNN + S	98.22	6,018	24.10	5.5×	99.68
VGG16 (baseline)	88.96	14,888,650	59,554	–	–
VGG16 + S	85.72	3,703,296	14,813	4×	96.36
1DCNN-GN (baseline)	99.90	3,719	14.53	–	–
1DCNN-GN + KD+QAP	97.71	397	1.50	9.4×	97.81
1DCNN-GN + S	99.90	168	0.66	22×	100.00

Table I summarizes the compression results. Across all evaluated architectures, *SNeuron* provides substantial parameter and storage reduction while preserving predictive accuracy.

For LeNet, *SNeuron* reduces model size from 170 KB to 1.97 KB ($88\times$ compression) while retaining 97.06% of baseline accuracy, enabling full deployment within on-chip and eliminating energy-intensive off-chip memory access. In contrast, KD+QAP achieves a $15.7\times$ compression ratio at the cost of a 6.4-point accuracy drop. For TinyCNN, *SNeuron* achieves $5.5\times$ compression with no measurable accuracy loss (99.68% retention), while KD+QAP reduces accuracy by more than 11 points. For VGG16, *SNeuron* compresses a 59.5 MB model to 14.8 MB ($4\times$) while retaining 96.36% of baseline accuracy, demonstrating that learnable gating scales effectively to deeper convolutional architectures. For 1DCNN-GN, *SNeuron* achieves $22\times$ compression while retaining 100% of baseline accuracy, reducing the model from 14.53 KB to 0.66 KB. This result is particularly notable as it demonstrates that *SNeuron* can identify an extremely compact subnetwork without any measurable accuracy loss, even in a real-world radiation detection application. In contrast, KD+QAP incurs a 2.19-point accuracy drop at only $9.4\times$ compression.

While the Lottery Ticket Hypothesis discovers sparse subnetworks through iterative pruning and retraining [12], *SNeuron* learns sparse structure directly during optimization via differentiable gating, without post-hoc extraction or fine-tuning.

VIII. FPGA Validation on Gamma/Neutron Classification

To validate deployment in resource-constrained environments, the proposed *SNeuron* was evaluated on a real-world gamma/neutron discrimination dataset [19]. The compressed models were synthesized to RTL using *hls4ml* [20] and deployed on a SoC-FPGA (xczu3eg-sfvc784-1-e) platform.

Table II reports the compression–accuracy trade-off on the gamma/neutron discrimination task. KD-based baselines retain approximately 400 parameters, achieving 74.58% accuracy with QAP and 96.99% with quantization-aware training (QAT). In contrast, the proposed *SNeuron* reduces the parameter count to approximately 130 while maintaining competitive predictive performance. Notably, *SNeuron* combined with 16-bit QAT achieves the highest accuracy (97.72%) using nearly three times fewer parameters than KD-based approaches.

FPGA synthesis results in Table III further demonstrate substantial reductions in DSP utilization relative to KD+QAT, indicating lower arithmetic complexity and reduced dynamic energy demand during inference.

TABLE II
COMPRESSION–ACCURACY TRADE-OFF ON THE GAMMA/NEUTRON CLASSIFICATION TASK.

Model	Parameters	Accuracy (%)
KD + QAP (8-bit, 20% sparsity)	≈400	74.58
KD + QAT (16-bit)	≈400	96.99
SNeuron	≈130	87.49
SNeuron + QAT (16-bit)	≈130	97.72

TABLE III
FPGA RESOURCE UTILIZATION (%) AND INFERENCE LATENCY FOR THE EVALUATED MODELS AFTER HLS-BASED SYNTHESIS AND DEPLOYMENT ON THE TARGET SOC-FPGA PLATFORM.

Model	LUT	LUTRAM	FF	BRAM	DSP	BUFG	Latency (ns)
KD + QAP (8-bit, 20%)	42	1	28	1	3	7	5.64×10^3
KD + QAT (16-bit)	40	1	12	1	62	2	2.61×10^4
SNeuron	44	1	25	1	1	5	7.53×10^3

The results highlight three key observations. First, *SNeuron* preserves competitive classification accuracy, with *SNeuron* + 16-bit QAT achieving the lowest overall misclassification. Second, FPGA measurements show a substantial DSP reduction (62% to 1%) and a $3.5\times$ latency improvement relative to KD+QAT, with a modest LUT increase (40% to 44%) that poses no practical constraint. Third, all gains are achieved through a single end-to-end training process, eliminating the multi-stage pipelines required by conventional compression methods. Together, these findings demonstrate that learnable neuron gating can simultaneously achieve accuracy preservation and hardware efficiency, making *SNeuron* well suited for embedded radiation sensing and other resource-constrained edge AI applications.

IX. CONCLUSION

This paper introduced *SNeuron*, a learnable gating mechanism that enables end-to-end structural sparsity for resource-constrained neural inference. Experimental results across four architectures and a real-world gamma/neutron discrimination task demonstrate consistent accuracy preservation alongside substantial reductions in parameter count and model size. FPGA implementation results further confirm reduced resource utilization, indicating lower arithmetic complexity. By discovering compact task-relevant subnetworks directly during gradient-based training, *SNeuron* provides a simple, hardware-friendly compression strategy suitable for edge AI deployment.

Future work will extend the proposed gating mechanism to residual and transformer-based architectures, and include systematic benchmarking on microcontroller platforms, targeting deployment on ultra-low-power devices.

ACKNOWLEDGMENT

The authors would like to thank the ICTP TRIL-KAUST program for the opportunity to conduct this research. The guidance of colleagues and mentors during the fellowship was invaluable in shaping this work.

REFERENCES

[1] C. Tian, T. Cheng, Z. Peng, W. Zuo, Y. Tian, Q. Zhang, F.-Y. Wang, and D. Zhang, "A survey on deep learning fundamentals," *Artificial Intelligence Review*, vol. 58, no. 12, p. 381, 2025.

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *3rd International Conference on Learning Representations (ICLR 2015)*. Computational and Biological Learning Society, 2015.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[4] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[5] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 6848–6856.

[6] N. D. Lane, S. Bhattacharya, A. Mathur, P. Georgiev, C. Forlivesi, and F. Kawsar, "Squeezing deep learning into mobile and embedded devices," *IEEE Pervasive Computing*, vol. 16, no. 3, pp. 82–88, 2017.

[7] I. Lamaakal, C. Yahyati, I. Ouahbi, K. El Makkaoui, and Y. Maleh, "A survey of model compression techniques for tinyml applications," in *2025 International Conference on Circuit, Systems and Communication (ICCSC)*. IEEE, 2025, pp. 1–6.

[8] T. Aarrestad, V. Loncar, N. Ghielmetti, M. Pierini, S. Summers, J. Ngadiuba, C. Petersson, H. Linander, Y. Iiyama, G. Di Guglielmo, J. Duarte, P. Harris, D. Rankin, S. Jindariani, K. Pedro, N. Tran, M. Liu, E. Kreinar, Z. Wu, and D. Hoang, "Fast convolutional neural networks on fpgas with hls4ml," *Machine Learning: Science and Technology*, vol. 2, no. 4, 7 2021.

[9] T. Choudhary, V. Mishra, A. Goswami, and J. Sarangapani, "A comprehensive survey on model compression and acceleration," *Artif. Intell. Rev.*, vol. 53, no. 7, pp. 5113–5155, 2020.

[10] S. Vadera and S. Ameen, "Methods for pruning deep neural networks," *Ieee Access*, vol. 10, pp. 63 280–63 300, 2022.

[11] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.

[12] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," *arXiv preprint arXiv:1803.03635*, 2018.

[13] R. S. Molina, I. R. Morales, M. L. Crespo, V. G. Costa, S. Carrato, and G. Ramponi, "An end-to-end workflow to efficiently compress and deploy dnn classifiers on soc/fpga," *IEEE Embedded Systems Letters*, vol. 16, no. 3, pp. 255–258, 2023.

[14] D. Ghimire, D. Kil, S. Jeong, J. Park, and S.-h. Kim, "One-cycle structured pruning via stability-driven subnetwork search," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2026, pp. 5467–5476.

[15] M. Xia, Z. Zhong, and D. Chen, "Structured pruning learns compact and accurate models," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 1513–1528.

[16] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 2002.

[17] L. Deng, "The mnist database of handwritten digit images for machine learning research," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.

[18] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[19] I. R. Morales, M. L. Crespo, M. Bogovac, A. Cicuttin, K. Kanaki, and S. Carrato, "Gamma/neutron classification with sipm clyc detectors using frequency-domain analysis for embedded real-time applications," *Nuclear Engineering and Technology*, vol. 56, no. 2, pp. 745–752, 2024.

[20] J.-F. Schulte, B. Ramhorst, C. Sun, J. Mitrevski, N. Ghielmetti, E. Lupi, D. Danopoulos, V. Loncar, J. Duarte, D. Burnette *et al.*, "hls4ml: A flexible, open-source platform for deep learning acceleration on reconfigurable hardware," *arXiv preprint arXiv:2512.01463*, 2025.